

ALL Operator in SQL Server with Examples

In this article, I am going to discuss the **ALL Operator in SQL Server** with Examples. Please read our previous article where we discussed the **IN, BETWEEN, and LIKE Operator** in detail. When a scalar value has to be compared with a single-column set of values then generally we use IN or JOINS. In addition to that, we can extend the comparison by using ANY and ALL operators which have rarely been used.

What is ALL Operator in SQL Server?

The ALL Operator in SQL Server is used to select all records of a Select statement. It compares a value to every value in a list of results from a query. The ALL must be preceded by the comparison operators and evaluates to TRUE if the query returns no rows. For example, ALL means greater than every value means greater than the maximum value. Suppose ALL (1, 2, 3) means greater than 3. If this is not clear at the moment don't worry we will try to understand this with some examples.

Understanding the ALL Operator with Examples:

Let us understand the SQL Server ALL Operator with some examples. We are going to use the following PermanentEmployee and ContractEmployee tables to understand the ALL Operator.

PermanentEmployee					ContractEmployee				
EmployeeId	Name	Gender	Department	Age	EmployeeId	Name	Gender	Department	Age
1	Pranaya	Male	IT	20	9	Anurag	Male	IT	33
2	Priyanka	Female	IT	22	10	Sara	Female	IT	15
3	Sudhanshu	Male	HR	25	11	Sambit	Male	HR	29
4	Subrat	Male	Sales	60	12	James	Male	Sales	37
5	Tarun	Male	Sales	54	13	Pam	Female	Sales	25
6	Lipika	Female	HR	27					
7	Smita	Female	IT	40					
8	Smith	Male	HR	29					

Please use the below script to create the database **EmployeeDB**, create **PermanentEmployee** and **ContractEmployee** table and populate these two tables with the required test data.

```
-- Create a database
CREATE DATABASE EmployeeDB
GO
Use EmployeeDB
GO
--Create PermanentEmployee Table
CREATE TABLE PermanentEmployee
(
    EmployeeId INT,
    Name VARCHAR(50),
    Gender VARCHAR(50),
    Department varchar(50),
```

```

    Age INT
)
GO
--Insert Rows into PermanentEmployee Table
INSERT INTO PermanentEmployee VALUES (1,'Pranaya','Male','IT',20)
INSERT INTO PermanentEmployee VALUES (2,'Priyanka','Female','IT',22)
INSERT INTO PermanentEmployee VALUES (3,'Sudhanshu','Male','HR',25)
INSERT INTO PermanentEmployee VALUES (4,'Subrat','Male','Sales',60)
INSERT INTO PermanentEmployee VALUES (5,'Tarun','Male','Sales',54)
INSERT INTO PermanentEmployee VALUES (6,'Lipika','Female','HR',27)
INSERT INTO PermanentEmployee VALUES (7,'Smita','Female','IT',40)
INSERT INTO PermanentEmployee VALUES (8,'Smith','Male','HR',29)
GO

```

ContractEmployee Table with test data:

```

-- Create ContractEmployee Table
CREATE TABLE ContractEmployee
(
    EmployeeId INT,
    Name VARCHAR(50),
    Gender VARCHAR(50),
    Department varchar(50),
    Age INT
)
GO
--Insert rows into ContractEmployee Table
INSERT INTO ContractEmployee VALUES (9,'Anurag','Male','IT',33)
INSERT INTO ContractEmployee VALUES (10,'Sara','Female','IT',15)
INSERT INTO ContractEmployee VALUES (11,'Sambit','Male','HR',29)
INSERT INTO ContractEmployee VALUES (12,'James','Male','Sales',37)
INSERT INTO ContractEmployee VALUES (13,'Pam','Female','Sales',25)
GO

```

Note: The ALL Operator in SQL Server can only be used with WHERE and HAVING clauses.

Syntax to use ALL Operator:

```

SELECT [column_name... | expression1 ]

FROM [table_name]

WHERE expression2 comparison_operator {ALL} (subquery)

```

Parameters:

1. **column_name**: Name of the column of the table.
2. **expression1**: Expression made up of a single constant, variable, scalar function, or column name and can also be the pieces of a SQL query that compare values against other values or perform arithmetic calculations.
3. **table_name**: Name of the table.
4. **WHERE expression2**: Compares a scalar expression, such as a column against every value in the subquery for ALL Operator. All rows must match the expression to return a Boolean TRUE value for the ALL operator.
5. **comparison_operator**: Compares the expression to the subquery. The comparison must be a standard comparison operator (=, <>, !=, >, >=, <, or <=).

Let's understand ALL Operator in SQL Server with Examples.

Consider a scenario where we have two tables such as PermanentEmployee and ContractEmployee. Both the tables have the column Age. If we need to get all the records from the PermanentEmployee table where the Age is greater than the maximum value of the Age column in the ContractEmployee table. What would be your query? We can use the subquery and max function to write our query for the above requirement as shown below.

SELECT * FROM PermanentEmployee WHERE Age > (SELECT MAX(AGE) FROM ContractEmployee)

The above query will produce the following result.

EmployeeId	Name	Gender	Department	Age
4	Subrat	Male	Sales	60
5	Tarun	Male	Sales	54
7	Smita	Female	IT	40

Let's see how to use the All Operator to get the same result.

For the above requirement, we can also use the ALL logical operator. In that case, we don't have to use the Max function. The ALL Operator is going to compare our outer query value to a set of values from the subquery. We can use >All, Greater than ALL means greater than every value returned by the subquery, In other words, greater than max value.

SELECT * FROM PermanentEmployee WHERE Age > ALL (SELECT AGE FROM ContractEmployee)

The above query will produce the same result as shown below.

EmployeeId	Name	Gender	Department	Age
4	Subrat	Male	Sales	60
5	Tarun	Male	Sales	54
7	Smita	Female	IT	40

With ALL we can use different comparison operators such as = , <> , != , > , >= , != , < , <= , !=

Let's consider ALL Operator with one more example.

Create a temporary table and populate the temporary table with some test data as shown below.

```
CREATE TABLE #TEMP_TABLE (ID INT)

INSERT INTO #TEMP_TABLE VALUES(1)
INSERT INTO #TEMP_TABLE VALUES(2)
INSERT INTO #TEMP_TABLE VALUES(3)
```

Example1:

```
IF 4 > ALL (SELECT ID FROM #TEMP_TABLE)
    PRINT 'Returned True'
ELSE
    PRINT 'Returned False'
```

In the above query 4 is greater than all of 1, 2, and 3. So 4, when compared with all of 1, 2, and 3 we can find 4, is greater. Hence the result is true.

Example2:

```
IF 3 > ALL (SELECT ID FROM #TEMP_TABLE)
    PRINT 'Returned True'
ELSE
    PRINT 'Returned False'
```

In the above query 3, greater is than all of 1, 2, and 3. So 3, when compared with all of 1, 2, and 3 we can find 3, is greater than 1 and 2 but not greater than 3. So 3 is not greater than all of 1, 2, and 3. Hence the result is false.

In the next article, I am going to discuss [ANY Operator in SQL Server](#) with Examples. Here, In this article, I try to explain the **ALL Operator in SQL Server** with Examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this ALL Operator in SQL Server with Examples article.